RAG in plain English

Written for a cloud / infra person, not a developer · a 10-minute read before an interview

1 The problem

An **LLM** (ChatGPT, Claude, DeepSeek…) is like a very well-read new joiner. It read the whole internet up to a date, but:

- it doesn't know **recent** things (its *knowledge cut-off*)
- it has **never seen your company's data**: Confluence, runbooks, HR policy
- when it doesn't know, it often makes things up confidently, which is called **hallucination**

Example: *"How many leaves do I get?"* The model has no idea which company you mean, or what its policy says.

2 Two ways to fix it

Fine-tuning

Re-train the model on your data, so the knowledge goes **inside** the model.

🔧 Like **re-baking the golden AMI** every time one config line changes. Slow and costly.

Use for: changing behaviour or tone.

RAG ✅

Leave the model alone. At question time, **look up** the right pages and hand them to it.

🔧 Like **mounting a ConfigMap at runtime**: update the doc, no rebuild.

Use for: company knowledge that changes.

3 RAG = Retrieval · Augmented · Generation

Retrieve the relevant pieces of your data → Augment (staple them to the question as **context**) → the LLM Generates the answer from them. The model never changes. **Only its input does.** It's like an *open-book exam* instead of answering from memory.

4 Two pipelines + one database in the middle

Indexing offline, once per document, like a CI build job



Querying online, every question, like the live request path



The build job **writes** into the vector DB and the live path **reads** from it. Two kinds of model are involved: an embedding model for finding, and an LLM for writing.

Golden rule 🔍

Wrong answer? **First check retrieval**: did the right chunk even reach the LLM?

🔧 Like checking the upstream before blaming the app.

Don't dump everything 🚫

Sending all 100 pages costs **$ per token**, it's **slower**, and the one fact gets **buried**, which leads to hallucination. Send only the **best 2–3 chunks**.

AI words, decoded 📖

Each word gets its meaning in one line, plus 🔧 an infra analogy to help it stick

① Talking to the model

LLM (Large Language Model): AI that reads text and writes text. GPT, Claude, DeepSeek.
🔧 A very well-read on-call engineer with no access to your wiki.

Token: a word-piece (~¾ of a word). Models read, limit and **bill** in tokens.
🔧 Like billable request units on a cloud bill.

Context window: the max tokens the model can see at once (e.g. 128K, 1M).
🔧 Like RAM: go past it and things fall out.

Prompt / Context: what you send: the question + instructions + retrieved pages (the *context*).

Hallucination: a confident but made-up answer.
🔧 Like a stale DNS cache: fast, sure, and wrong.

Streaming: the answer arrives word by word instead of all at the end.
🔧 `tail -f` vs downloading the whole log file.

OpenRouter: one gateway and one key in front of hundreds of models. Swap models by name. It has Compare and Logs pages.
🔧 An API gateway / load balancer in front of many backends.

API key + credit limit: the password to call models, with a $ cap.
🔧 An IAM access key with a billing alarm.

SDK: a library to call a service from code (we used the *openai* SDK).
🔧 Like boto3 for AWS.

② Preparing documents

Parsing: turn a PDF, image or wiki page into clean text.
🔧 Log parsing: raw blob → structured lines.

Chunking: cut the text into small pieces (by size, paragraph, meaning…).
🔧 Like splitting one giant log into rotated files. A card number reads easier as 4128-0012-3456-7890.

Indexing vs Querying: the offline prep job vs the live per-question path.
🔧 Build pipeline vs runtime traffic.

③ Embeddings (the magic part)

Embedding: text turned into a list of numbers that captures its **meaning**.
🔧 Like a hash, except similar text gets similar numbers (a hash does the opposite).

Vector: just an array of numbers: [0.12, −0.31, 0.81 …].

Dimension: how many numbers are in the vector (384 … 3072). Food example: [spicy, sweet] = 2D, add crunchy = 3D.
🔧 Like the number of metrics you track per server.

Embedding model: the model that makes vectors (text-embedding-3-small = 1536, NVIDIA nemotron-embed = 2048). **Use the same model for docs and questions!**

Semantic: about meaning, not exact words. "PTO" matches "leaves". *bank* + river ≠ *bank* + money.

Tokenizer → Transformer: chops text into tokens, then **attention** layers let each word look at its neighbours to get the meaning in context.

Normalise: scale a vector to length 1, so only its direction (meaning) counts.

Search & vector DB words



How we find the right chunk, and where the chunks live

④ Measuring "how close"

Similarity / distance metric: the rule that decides how "close" two vectors are.
🔧 Like choosing the SLO metric before saying which server is "best".

Cosine similarity: the angle between two arrows. 1 = same, 0 = unrelated, −1 = opposite. **Higher = better.** The default for text and RAG.

Euclidean (L2): straight-line distance. **Lower = better.** Used for photos and coordinates.

Dot product: alignment + size. The same ranking as cosine when vectors are normalised, and the fastest.

Manhattan (L1): city-block distance. One glitchy value can't dominate the score.

Top-K: return the K best matches.
🔧 `sort -r | head -n K`

Score threshold: drop matches below a score (e.g. 0.5). Top-K always returns K, even junk.
🔧 Like an alert threshold: it kills the noise.

Recall: of all the truly relevant chunks, how many did we find? (3 of 4 = 75%)
🔧 Of all real incidents, how many fired an alert?

Precision: of what we returned, how many were useful? (3 of 10 = 30%)
🔧 Of all alerts fired, how many were real?

⑤ Vector database

Vector database: stores vectors + text, searches fast, filters, persists and scales. Qdrant, pgvector, Pinecone.
🔧 A purpose-built DB, just as Prometheus is for metrics.

Qdrant: an open-source vector DB (Rust). `:memory:` for learning → Docker → cloud. Ports **6333** REST + dashboard, **6334** gRPC.

Collection: like a **table**, with a fixed vector size and metric.

Point / Payload: one row = id + vector + payload (JSON: text, topic, source).

Upsert: insert, or update if the id exists. Safe to re-run.
🔧 Idempotent, like `kubectl apply`.

Exact search: compare the question with every vector. Perfect, but slow at millions.
🔧 A full table scan.

ANN (Approximate Nearest Neighbour): use an index to check only likely candidates. Much faster, and rarely misses one.

HNSW: Qdrant's ANN index, a layered graph: highway → main road → street.
🔧 Like DNS lookup: root → .com → domain.

IVF: the other ANN family. Group vectors into clusters and search only the nearest clusters.
🔧 Like routing to the nearest region first.

DevOps catch from class: Qdrant in Docker without a volume loses its data when the container is re-created. Always run

```
docker run -p 6333:6333 -p 6334:6334 -v
qdrant_storage:/qdrant/storage qdrant/qdrant
```

Before the interview

Read this page last. Say it out loud once.

"Explain RAG" in 30 seconds

"LLMs don't know our private or recent data, and they hallucinate when they're unsure. With RAG we **index** our documents offline: parse, chunk, turn the chunks into **embeddings** and store them in a **vector DB** like Qdrant. At question time we embed the question, find the **closest chunks** with cosine similarity, and pass the top few to the LLM as **context**. Updating knowledge means re-indexing documents. We never retrain the model."

Quick answers

RAG or fine-tuning?

Needs **information** → RAG. Needs to **behave** differently → fine-tune.

Switched embedding model?

Re-embed and re-index everything into a new collection. Never mix models.

Bot gave a wrong answer?

Check **retrieval first**: was the right chunk in the top-K?

Classic similarity bug?

Reading it backwards: cosine **high** = good, L2 **low** = good.

Why not send all the docs?

Cost per token, latency, and the fact gets buried → hallucination.

Top-K problem?

It always returns K, even junk. Add a **score threshold** and say "I don't know".

What's an embedding?

A fixed-length vector of meaning. Similar text → nearby vectors.

Why a vector DB?

Brute force is too slow at millions. It gives an ANN index (HNSW), filters, persistence and scaling.

More dimensions = better?

Not by itself. More memory and $; chunking + model matter more. Measure recall.

Streaming vs non-streaming?

Word by word (feels fast) vs everything at the end.

Which metric?

Text / RAG chatbot	**Cosine**
Vectors normalised	**Dot** (fastest)
"Customers also liked"	**Dot** (popularity counts)
Similar photos	**Euclidean**
City delivery / glitchy sensors	**Manhattan**

Numbers to remember

1 dimension	4 bytes
1536-dim vector	≈ 6 KB
1M chunks × 1536	≈ 6 GB
Good cosine (our lab)	0.614 vs ≤0.2 junk
Qdrant ports	6333 · 6334

Chapter 01 ✓